ICML 2026

SimpleTool (RealtimeTool)

Parallel Decoding for Real-Time LLM Function Calling

Xiaoxin Shi[1,2], Jiaxin Wan[2], Linkang Dong[2], Wei Jiang[2], Yue Liu[2], Zengfeng Huang[2,3] · [1]Shanghai Jiao Tong University · [2]Shanghai Innovation Institute · [3]Fudan University

GitHub · arXiv · ICML · Hugging Face · ModelScope

open models · **0.5B → 30B-A3B** (dense + MoE)

Function calls in real time: 3–6× faster, competitive or better accuracy — 16 Hz on a consumer GPU.

THE REAL-TIME GAP

A function call must be complete before it can execute.

LLMs call tools by emitting **structured, schema-constrained output** — a function name and its arguments. But a partial call is meaningless, so a tool call has to be timed **end-to-end** — it can't hide latency behind streaming the way chat does. Real-time control (embodied AI, game AI, avatars) needs **5–30 Hz**, yet on-device VLA and LLM-agent loops are still slow: the function call **is** the whole output and must land within one control cycle.

KEY INSIGHT

An API call's arguments barely depend on each other's order — unlike words in a sentence. Strict causal, left-to-right decoding is unnecessary, so name and arguments can be produced in parallel.

THE METHOD

1 · Special tokens (dual role)
Absorb structure into markers — **4–6× fewer tokens** — and act as **mode selectors** choosing which component to emit (17 tokens: `<function>`, `<argk>`, `<null>`).

2 · Parallel decoding
Function name and each argument become **independent streams** sharing one prefix KV cache — only the appended token differs. Dependent params fold into a single head.

3 · LoRA adaptation
A small **LoRA** run adapts the model to this format — no architecture change, fast convergence, and the base model's **general abilities stay intact** (Appendix).

$$T_e \approx T_p + \max_i(N_i)\cdot T_d$$

T_e = tool-use end-to-end latency — the longest head, not the sum ($\Sigma\ N\cdot T_d$)

WHY IT'S NEARLY FREE — AND THE RESULT

At **batch 1**, decoding is **memory-bandwidth-bound** and compute sits idle. Parallel heads fill that idle compute, so 8 heads add just **+8.2%** (93% efficiency) — parallelism is nearly free.

3–6× end-to-end speedup, **up to 9.6×** — no draft model

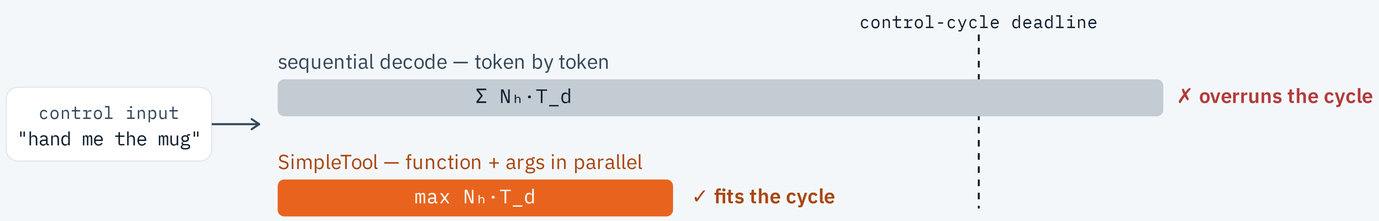
61.2 ms P50 at 4B, AWQ on RTX 4090 = **16 Hz**

5 / 5 benchmarks **competitive or higher** (avg +2.9%)

DROP-IN

Runs as-is on existing serving stacks — vLLM · SGLang · llama.cpp, no inference-framework changes.

WORKED EXAMPLE — ONE EMBODIED CONTROL CYCLE



Same call every time — `grasp(object=mug, hand=left, force=gentle)` — but only **parallel decoding** returns it before the next control tick.

ON A REAL IPHONE

A 0.5B model, real-time on-device.

iPhone 17 Pro Max · llama.cpp · INT8 · vs FunctionGemma-270M

FASTER	P50 **528 ms** vs 709 ms · P90 **1018** vs 1570 ms
MORE ACCURATE	**86.2%** vs 85.0% on Mobile Actions
STOCK BACKEND	plain llama.cpp, **no kernel-level optimization** — headroom remains

WHERE IT GOES

Real-time **embodied AI**, **game AI & NPCs**, interactive avatars, and on-device agents — any loop where the function call is the control output and must land in one cycle.

WHAT'S NEXT

VLM backbone → real-time vision-language-action · **Windows & iOS** on-device (llama.cpp) · tracking **AI-Native games** & **embodied intelligence** · composes with speculative decoding & layer-skip.

Xiaoxin Shi
cialtion737410@sjtu.edu.cn
code · models · paper — scan above